

CAHIER DES CHARGES

SYSTÈME DE GESTION DE LIBRAIRIE

1. PRÉSENTATION DU PROJET

1.1 Contexte

Développement d'une application web de gestion complète pour librairie physique, permettant la gestion des stocks, des ventes, des utilisateurs et la génération de rapports.

1.2 Objectifs

- Automatiser la gestion des stocks de livres
- Faciliter le processus de vente avec génération automatique de factures
- Assurer un suivi précis des ventes par vendeur
- Gérer les réductions et promotions
- Générer des rapports d'activité quotidiens et périodiques
- Alerter sur les stocks faibles

1.3 Périmètre

Inclus :

- Gestion des catégories et livres
- Gestion des utilisateurs (Admin/Vendeurs)
- Processus de vente avec réductions
- Génération de factures PDF
- Gestion des mouvements de stock
- Rapports et statistiques
- Alertes de stock
- Réservations clients

Exclus (Phase 1) :

- Gestion des modes de paiement (paiement manuel)
- Enregistrement des clients
- Gestion de fidélité

- Multi-magasins
- Application mobile native

2. SPÉCIFICATIONS FONCTIONNELLES

2.1 GESTION DES UTILISATEURS

2.1.1 Authentification

- **Inscription** : Réservée aux administrateurs uniquement
- **Connexion** : Email + Mot de passe avec JWT
- **Déconnexion** : Invalidation du token
- **Sécurité** :
 - Mots de passe hashés (bcrypt)
 - Tokens JWT avec expiration
 - Refresh tokens
 - Limitation des tentatives de connexion (5 max)

2.1.2 Rôles et Permissions

Administrateur :

- Accès total à toutes les fonctionnalités
- Gestion des utilisateurs (CRUD)
- Gestion des catégories et livres
- Gestion des réductions
- Consultation et génération de tous les rapports
- Ajustement des stocks
- Annulation de ventes
- Configuration système

Vendeur :

- Consultation du catalogue (livres et catégories)
- Création de ventes
- Application de réductions actives
- Génération de factures
- Consultation de ses propres statistiques
- Création de réservations clients

2.1.3 Profil Utilisateur

- Nom complet
- Email (unique)
- Rôle (admin/vendeur)
- Date de création
- Statut (actif/inactif)
- Dernière connexion
- Statistiques personnelles (vendeurs)

2.2 GESTION DES CATÉGORIES

2.2.1 Attributs

- **Nom** : Obligatoire, unique (ex: "Romans", "Informatique")
- **Description** : Optionnel, texte libre
- **Code** : Auto-généré (4 premières lettres du nom + 3 chiffres aléatoires)
 - Exemple : "Romans" → "ROMA123", "Informatique" → "INFO456"
 - Gestion des doublons : Si "ROMA123" existe, générer "ROMA124"

2.2.2 Fonctionnalités

- Création, modification, suppression (si aucun livre associé)
- Désactivation (si livres associés)
- Liste paginée avec recherche
- Statistiques par catégorie (nombre de livres, CA généré)

2.3 GESTION DES LIVRES

2.3.1 Attributs

- **Titre** : Obligatoire
- **Description** : Optionnel
- **Auteur** : Obligatoire
- **Maison d'édition** : Obligatoire
- **Date de parution** : Format date
- **Langue** : Liste prédéfinie (Français, Anglais, Espagnol, etc.)
- **ISBN** : Optionnel (format international standard)
- **Quantité en stock** : Nombre entier ≥ 0

- **Seuil minimal** : Alerte de stock bas (défaut : 5)
- **Prix de vente** : Obligatoire, décimal (XAF ou devise locale)
- **Prix d'achat** : Optionnel (pour calcul de marge)
- **Emplacement** : Optionnel (rayon/étagère physique)
- **Catégorie** : Obligatoire (liaison avec table catégories)
- **Code** : Auto-généré (Code catégorie + tiret + 3 chiffres)
 - Exemple : Livre dans "ROMA123" → "ROMA123-001"
- **Statut** : Actif/Inactif
- **Date d'ajout** : Auto

2.3.2 Fonctionnalités

CRUD Complet :

- Création avec validation des champs
- Modification (prix, quantité, seuil, etc.)
- Suppression logique (désactivation si historique de ventes)
- Liste paginée (20 par page)

Recherche Avancée :

- Par titre (recherche partielle)
- Par auteur
- Par code
- Par catégorie
- Par ISBN
- Par statut de stock (disponible/stock bas/rupture)
- Recherche multi-critères

Gestion du Stock :

- Ajout de stock (réapprovisionnement)
- Ajustement manuel (inventaire)
- Déduction automatique lors des ventes
- Historique des mouvements

Alertes :

- Badge visuel si quantité $\leq$ seuil minimal
- Liste des livres en stock critique
- Notification pour l'admin

Import/Export :

- Import Excel (création en masse)
- Export Excel (inventaire complet)

2.4 GESTION DES FOURNISSEURS

2.4.1 Attributs

- Nom
- Contact (personne responsable)
- Téléphone
- Email
- Adresse
- Délai de livraison (en jours)
- Statut (actif/inactif)

2.4.2 Fonctionnalités

- CRUD complet
- Liste avec recherche
- Historique des commandes passées

2.5 GESTION DES COMMANDES FOURNISSEURS

2.5.1 Attributs Commande

- Numéro de commande (auto-généré : CMD-YYYYMMDD-XXX)
- Fournisseur
- Date de commande
- Date de réception prévue
- Date de réception réelle
- Statut (en_attente/reçue/annulée)
- Montant total
- Notes

2.5.2 Lignes de Commande

- Livre
- Quantité commandée
- Prix d'achat unitaire
- Sous-total

2.5.3 Workflow

1. Admin crée une commande
2. Ajoute les livres et quantités
3. Valide la commande → statut "en_attente"
4. À réception → marque comme "reçue"
5. Stock des livres mis à jour automatiquement
6. Mouvement de stock enregistré

2.6 GESTION DES RÉDUCTIONS

2.6.1 Attributs

- **Intitulé** : Obligatoire (ex: "Promo Rentrée", "Soldes d'été")
- **Description** : Optionnel
- **Type** :
 - Pourcentage (ex: 15%)
 - Montant fixe (ex: 1000 XAF)
- **Valeur** : Nombre (pourcentage ou montant)
- **Cible** :
 - Globale (s'applique à toutes les ventes)
 - Livre spécifique
 - Catégorie entière
- **Cible ID** : ID du livre ou catégorie si applicable
- **Date de début** : Obligatoire
- **Date de fin** : Obligatoire
- **Actif** : Boolean (peut être désactivée manuellement)

2.6.2 Règles d'Application

- Une seule réduction par livre dans une vente
- Priorité : Réduction livre > Réduction catégorie > Réduction globale
- Vérification automatique de la période de validité
- Application automatique lors de l'ajout au panier

2.6.3 Fonctionnalités

- CRUD complet
- Activation/désactivation rapide
- Liste des réductions actives/expirées

- Statistiques d'utilisation

2.7 GESTION DES VENTES

2.7.1 Processus de Vente

Étape 1 : Création du panier

- Vendeur recherche et ajoute des livres
- Quantité modifiable pour chaque ligne
- Vérification du stock disponible en temps réel
- Application automatique des réductions actives
- Calcul automatique des sous-totaux et total

Étape 2 : Validation

- Vendeur valide la vente
- Génération automatique du numéro de facture
- Enregistrement de la vente
- Déduction automatique du stock
- Enregistrement des mouvements de stock

Étape 3 : Facturation

- Génération automatique de la facture PDF
- Affichage pour impression
- Option de téléchargement

2.7.2 Structure Vente

Table Vente (Transaction principale) :

- ID
- Numéro de facture : FAC-YYYYMMDD-XXX (ex: FAC-20260105-001)
- Date et heure
- Vendeur (user_id)
- Montant total HT
- Montant des réductions
- Montant total TTC
- Statut (validée/annulée)

Table Lignes_Vente (Détail articles) :

- ID
- Vente ID
- Livre ID
- Titre du livre (snapshot)
- Prix unitaire
- Quantité
- Réduction appliquée (ID et montant)
- Sous-total

2.7.3 Annulation de Vente

- Réservée à l'admin
- Restauration automatique du stock
- Création de mouvements de stock "retour"
- Vente marquée comme "annulée" (pas de suppression)
- Motif d'annulation obligatoire

2.7.4 Facture PDF

Contenu :

- En-tête : Logo, nom de la librairie, coordonnées
- Numéro de facture et date
- Nom du vendeur
- Tableau des articles :
 - Code livre
 - Titre
 - Prix unitaire
 - Quantité
 - Réduction
 - Sous-total
- Total HT
- Total des réductions
- TVA (si applicable)
- Total TTC
- Pied de page : Mentions légales, "Merci de votre visite"

2.8 GESTION DES MOUVEMENTS DE STOCK

2.8.1 Types de Mouvements

- **ENTREE** : Réapprovisionnement, réception commande
- **SORTIE** : Vente (automatique)
- **AJUSTEMENT** : Inventaire, correction, perte, don
- **RETOUR** : Annulation de vente

2.8.2 Attributs

- ID
- Livre
- Type de mouvement
- Quantité (+/-)
- Stock avant
- Stock après
- Motif/Description
- Référence (numéro facture, numéro commande, etc.)
- Utilisateur
- Date et heure

2.8.3 Traçabilité

- Chaque modification de stock génère un mouvement
- Historique complet consultable
- Filtres par livre, type, période, utilisateur

2.9 RÉSERVATIONS CLIENTS

2.9.1 Attributs

- ID
- Nom du client
- Téléphone
- Livre demandé (texte libre ou ID livre si existe)
- Date de réservation
- Date de disponibilité prévue
- Statut (en_attente/disponible/récupérée/annulée)
- Notes
- Créé par (vendeur)

2.9.2 Fonctionnalités

- Création rapide par vendeur
- Liste des réservations en attente
- Notification quand livre disponible
- Conversion en vente directe
- Historique des réservations

2.10 RAPPORTS ET STATISTIQUES

2.10.1 Rapport de Clôture Journalière

Informations :

- Date du rapport
- Nombre total de ventes
- Chiffre d'affaires total
- Montant total des réductions accordées
- Nombre de livres vendus (quantité)
- CA par vendeur
- Top 5 des livres vendus
- Top 3 des catégories
- Alertes stock (livres sous seuil)

Format : PDF téléchargeable + Affichage web

2.10.2 Rapports Périodiques

- Hebdomadaire
- Mensuel
- Annuel
- Période personnalisée

Contenu :

- Évolution du CA (graphique)
- Comparaison avec période précédente
- Performance par vendeur
- Performance par catégorie
- Livres les plus vendus
- Analyse des réductions

- Rotation des stocks
- Marge bénéficiaire (si prix d'achat renseigné)

2.10.3 Tableaux de Bord

Dashboard Admin :

- CA du jour/mois
- Nombre de ventes du jour
- Stock critique (nombre de livres)
- Graphique évolution CA (7 derniers jours)
- Top 5 livres du mois
- Performance des vendeurs
- Alertes et notifications

Dashboard Vendeur :

- Mes ventes du jour
- Mon CA du jour/mois
- Mes meilleures ventes
- Livres en stock critique

2.10.4 Exports

- Export Excel de tous les rapports
- Format CSV pour imports externes
- Export inventaire complet

2.11 ALERTES ET NOTIFICATIONS

2.11.1 Types d'Alertes

Stock Critique :

- Badge rouge sur les livres (quantité $\leq$ seuil)
- Liste centralisée des livres à commander
- Notification admin (quotidienne)

Activité Système :

- Nouvelle commande reçue
- Réservation client en attente
- Échec de connexion répétés (sécurité)

Performances :

- Objectif de vente atteint
- Alerte CA faible (optionnel)

2.11.2 Centre de Notifications

- Liste des notifications non lues
- Marquage lu/non lu
- Archivage
- Filtres par type

3. SPÉCIFICATIONS TECHNIQUES

3.1 ARCHITECTURE

3.1.1 Type d'Architecture

Architecture 3-tiers :

- **Frontend** : Interface utilisateur (SPA)
- **Backend** : API REST
- **Base de données** : Relationnelle

3.1.2 Stack Technologique Recommandée

Backend :

- **Langage** : Node.js (JavaScript/TypeScript) ou Python (FastAPI/Django)
- **Framework** :
 - Option 1 : Express.js (Node.js)
 - Option 2 : FastAPI (Python)
 - Option 3 : NestJS (TypeScript - plus structuré)
- **ORM** :
 - Prisma (Node.js)
 - SQLAlchemy (Python)
 - TypeORM (TypeScript)
- **Authentication** : JWT (jsonwebtoken)
- **Validation** : Joi, Zod, ou Pydantic
- **Génération PDF** : PDFKit, jsPDF, ou ReportLab

Frontend :

- **Framework :**
 - Option 1 : React.js + Vite
 - Option 2 : Vue.js 3 + Vite
 - Option 3 : Next.js (React avec SSR)
- **UI Library :**
 - Tailwind CSS + shadcn/ui
 - Material-UI (MUI)
 - Ant Design
- **State Management :**
 - React Query + Context API (React)
 - Pinia (Vue)
- **Routing :** React Router ou Vue Router
- **Formulaires :** React Hook Form ou Formik
- **Graphiques :** Recharts, Chart.js, ou ApexCharts

Base de Données :

- **SGBD :** PostgreSQL (recommandé) ou MySQL
- **Alternative :** SQLite (développement uniquement)

Outils Complémentaires :

- **API Documentation :** Swagger/OpenAPI
- **Testing :** Jest, Vitest, Pytest
- **Logs :** Winston, Pino
- **Emails :** Nodemailer (notifications optionnelles)

3.2 BASE DE DONNÉES

3.2.1 Schéma Relationnel

```
-- USERS
users (
  id: UUID PRIMARY KEY,
  nom_complet: VARCHAR(100) NOT NULL,
  email: VARCHAR(100) UNIQUE NOT NULL,
  password_hash: VARCHAR(255) NOT NULL,
  role: ENUM('admin', 'vendeur') NOT NULL,
  statut: ENUM('actif', 'inactif') DEFAULT 'actif',
  date_creation: TIMESTAMP DEFAULT NOW(),
  derniere_connexion: TIMESTAMP,
  created_at: TIMESTAMP DEFAULT NOW(),
  updated_at: TIMESTAMP DEFAULT NOW()
)
```

```

-- CATEGORIES
categories (
  id: UUID PRIMARY KEY,
  nom: VARCHAR(100) UNIQUE NOT NULL,
  description: TEXT,
  code: VARCHAR(10) UNIQUE NOT NULL,
  statut: ENUM('actif', 'inactif') DEFAULT 'actif',
  created_at: TIMESTAMP DEFAULT NOW(),
  updated_at: TIMESTAMP DEFAULT NOW()
)

-- LIVRES
livres (
  id: UUID PRIMARY KEY,
  code: VARCHAR(20) UNIQUE NOT NULL,
  titre: VARCHAR(255) NOT NULL,
  auteur: VARCHAR(100) NOT NULL,
  description: TEXT,
  maison_edition: VARCHAR(100) NOT NULL,
  date_parution: DATE,
  isbn: VARCHAR(20) UNIQUE,
  langue: VARCHAR(50) NOT NULL,
  quantite_stock: INTEGER DEFAULT 0 CHECK (quantite_stock >= 0),
  seuil_minimal: INTEGER DEFAULT 5,
  prix_vente: DECIMAL(10,2) NOT NULL,
  prix_achat: DECIMAL(10,2),
  emplacement: VARCHAR(50),
  categorie_id: UUID REFERENCES categories(id),
  statut: ENUM('actif', 'inactif') DEFAULT 'actif',
  created_at: TIMESTAMP DEFAULT NOW(),
  updated_at: TIMESTAMP DEFAULT NOW(),
  INDEX idx_code (code),
  INDEX idx_titre (titre),
  INDEX idx_categorie (categorie_id),
  INDEX idx_stock (quantite_stock, seuil_minimal)
)

-- FOURNISSEURS
fournisseurs (
  id: UUID PRIMARY KEY,
  nom: VARCHAR(100) NOT NULL,
  contact: VARCHAR(100),
  telephone: VARCHAR(20),
  email: VARCHAR(100),
  adresse: TEXT,
  delai_livraison_jours: INTEGER,
  statut: ENUM('actif', 'inactif') DEFAULT 'actif',
  created_at: TIMESTAMP DEFAULT NOW(),
  updated_at: TIMESTAMP DEFAULT NOW()
)

-- COMMANDES FOURNISSEURS

```

```

commandes_fournisseurs (
  id: UUID PRIMARY KEY,
  numero_commande: VARCHAR(30) UNIQUE NOT NULL,
  fournisseur_id: UUID REFERENCES fournisseurs(id),
  date_commande: DATE NOT NULL,
  date_reception_prevue: DATE,
  date_reception_reelle: DATE,
  montant_total: DECIMAL(10,2),
  statut: ENUM('en_attente', 'recue', 'annulee') DEFAULT 'en_attente',
  notes: TEXT,
  created_by: UUID REFERENCES users(id),
  created_at: TIMESTAMP DEFAULT NOW(),
  updated_at: TIMESTAMP DEFAULT NOW()
)

-- LIGNES COMMANDES FOURNISSEURS
lignes_commande_fournisseur (
  id: UUID PRIMARY KEY,
  commande_id: UUID REFERENCES commandes_fournisseurs(id) ON DELETE CASCADE,
  livre_id: UUID REFERENCES livres(id),
  quantite: INTEGER NOT NULL,
  prix_achat_unitaire: DECIMAL(10,2) NOT NULL,
  sous_total: DECIMAL(10,2) NOT NULL,
  created_at: TIMESTAMP DEFAULT NOW()
)

-- REDUCTIONS
reductions (
  id: UUID PRIMARY KEY,
  intitule: VARCHAR(100) NOT NULL,
  description: TEXT,
  type: ENUM('pourcentage', 'montant_fixe') NOT NULL,
  valeur: DECIMAL(10,2) NOT NULL,
  cible: ENUM('globale', 'livre', 'categorie') NOT NULL,
  cible_id: UUID,
  date_debut: DATE NOT NULL,
  date_fin: DATE NOT NULL,
  actif: BOOLEAN DEFAULT true,
  created_at: TIMESTAMP DEFAULT NOW(),
  updated_at: TIMESTAMP DEFAULT NOW(),
  CHECK (date_fin >= date_debut),
  INDEX idx_dates (date_debut, date_fin, actif)
)

-- VENTES
ventes (
  id: UUID PRIMARY KEY,
  numero_facture: VARCHAR(30) UNIQUE NOT NULL,
  date_vente: TIMESTAMP DEFAULT NOW(),
  vendeur_id: UUID REFERENCES users(id),
  montant_ht: DECIMAL(10,2) NOT NULL,
  montant_reductions: DECIMAL(10,2) DEFAULT 0,
  montant_ttc: DECIMAL(10,2) NOT NULL,

```

```

statut: ENUM('validee', 'annulee') DEFAULT 'validee',
motif_annulation: TEXT,
created_at: TIMESTAMP DEFAULT NOW(),
updated_at: TIMESTAMP DEFAULT NOW(),
INDEX idx_numero (numero_facture),
INDEX idx_date (date_vente),
INDEX idx_vendeur (vendeur_id)
)

-- LIGNES VENTE
lignes_vente (
  id: UUID PRIMARY KEY,
  vente_id: UUID REFERENCES ventes(id) ON DELETE CASCADE,
  livre_id: UUID REFERENCES livres(id),
  titre_livre: VARCHAR(255) NOT NULL,
  code_livre: VARCHAR(20) NOT NULL,
  prix_unitaire: DECIMAL(10,2) NOT NULL,
  quantite: INTEGER NOT NULL,
  reduction_id: UUID REFERENCES reductions(id),
  montant_reduction: DECIMAL(10,2) DEFAULT 0,
  sous_total: DECIMAL(10,2) NOT NULL,
  created_at: TIMESTAMP DEFAULT NOW()
)

-- MOUVEMENTS STOCK
mouvements_stock (
  id: UUID PRIMARY KEY,
  livre_id: UUID REFERENCES livres(id),
  type_mouvement: ENUM('entree', 'sortie', 'ajustement', 'retour') NOT NULL,
  quantite: INTEGER NOT NULL,
  stock_avant: INTEGER NOT NULL,
  stock_apres: INTEGER NOT NULL,
  motif: VARCHAR(255),
  reference: VARCHAR(50),
  user_id: UUID REFERENCES users(id),
  date_mouvement: TIMESTAMP DEFAULT NOW(),
  INDEX idx_livre (livre_id),
  INDEX idx_date (date_mouvement)
)

-- RESERVATIONS CLIENTS
reservations_clients (
  id: UUID PRIMARY KEY,
  nom_client: VARCHAR(100) NOT NULL,
  telephone: VARCHAR(20) NOT NULL,
  livre_demande: VARCHAR(255) NOT NULL,
  livre_id: UUID REFERENCES livres(id),
  date_reservation: DATE DEFAULT CURRENT_DATE,
  date_disponibilite_prevue: DATE,
  statut: ENUM('en_attente', 'disponible', 'recuperee', 'annulee') DEFAULT 'en_attente',
  notes: TEXT,
  cree_par: UUID REFERENCES users(id),
  created_at: TIMESTAMP DEFAULT NOW(),

```



```

    updated_at: TIMESTAMP DEFAULT NOW(),
    INDEX idx_statut (statut),
    INDEX idx_telephone (telephone)
)

-- NOTIFICATIONS
notifications (
    id: UUID PRIMARY KEY,
    user_id: UUID REFERENCES users(id),
    type: VARCHAR(50) NOT NULL,
    titre: VARCHAR(255) NOT NULL,
    message: TEXT NOT NULL,
    lu: BOOLEAN DEFAULT false,
    date_notification: TIMESTAMP DEFAULT NOW(),
    INDEX idx_user_lu (user_id, lu)
)

-- LOGS ACTIVITE (optionnel mais recommandé)
logs_activite (
    id: UUID PRIMARY KEY,
    user_id: UUID REFERENCES users(id),
    action: VARCHAR(100) NOT NULL,
    entite: VARCHAR(50),
    entite_id: UUID,
    details: JSONB,
    ip_address: VARCHAR(45),
    user_agent: TEXT,
    date_action: TIMESTAMP DEFAULT NOW(),
    INDEX idx_user (user_id),
    INDEX idx_date (date_action),
    INDEX idx_entite (entite, entite_id)
)

```

3.2.2 Indexes Recommandés

Déjà inclus dans le schéma ci-dessus pour optimiser les performances des requêtes fréquentes.

3.3 API REST

3.3.1 Structure des Endpoints

Authentification :

```

POST   /api/auth/login
POST   /api/auth/logout
POST   /api/auth/refresh-token
GET    /api/auth/me

```

Utilisateurs :

```
GET    /api/users (admin only)
POST   /api/users (admin only)
GET    /api/users/:id (admin only)
PUT    /api/users/:id (admin only)
DELETE /api/users/:id (admin only)
GET    /api/users/:id/stats
```

Catégories :

```
GET    /api/categories
POST   /api/categories (admin only)
GET    /api/categories/:id
PUT    /api/categories/:id (admin only)
DELETE /api/categories/:id (admin only)
GET    /api/categories/:id/stats
```

Livres :

```
GET    /api/livres
POST   /api/livres (admin only)
GET    /api/livres/:id
PUT    /api/livres/:id (admin only)
DELETE /api/livres/:id (admin only)
GET    /api/livres/stock-critique
POST   /api/livres/import (admin only)
GET    /api/livres/export (admin only)
GET    /api/livres/search?q=
```

Fournisseurs :

```
GET    /api/fournisseurs (admin only)
POST   /api/fournisseurs (admin only)
GET    /api/fournisseurs/:id (admin only)
PUT    /api/fournisseurs/:id (admin only)
DELETE /api/fournisseurs/:id (admin only)
```

Commandes Fournisseurs :

```
GET    /api/commandes (admin only)
POST   /api/commandes (admin only)
GET    /api/commandes/:id (admin only)
PUT    /api/commandes/:id (admin only)
POST   /api/commandes/:id/recevoir (admin only)
POST   /api/commandes/:id/annuler (admin only)
```

Réductions :

```
GET    /api/reductions
POST   /api/reductions (admin only)
GET    /api/reductions/:id
PUT    /api/reductions/:id (admin only)
DELETE /api/reductions/:id (admin only)
GET    /api/reductions/actives
PATCH /api/reductions/:id/toggle (admin only)
```

Ventes :

```
GET    /api/ventes
POST   /api/ventes
GET    /api/ventes/:id
POST   /api/ventes/:id/annuler (admin only)
GET    /api/ventes/:id/facture (PDF)
GET    /api/ventes/mes-ventes (vendeur)
```

Mouvements Stock :

```
GET    /api/mouvements (admin only)
POST   /api/mouvements (admin only)
GET    /api/mouvements/livre/:id
```

Réservations :

```
GET    /api/reservations
POST   /api/reservations
GET    /api/reservations/:id
PUT    /api/reservations/:id
DELETE /api/reservations/:id
PATCH /api/reservations/:id/statut
```

Rapports :

```
GET    /api/rapports/cloture-journaliere?date=
GET    /api/rapports/periode?debut=&fin=
GET    /api/rapports/top-livres?periode=
GET    /api/rapports/performance-vendeurs?periode=
GET    /api/rapports/dashboard
```

Notifications :

```
GET    /api/notifications
PATCH /api/notifications/:id/lire
DELETE /api/notifications/:id
```

3.3.2 Format des Réponses

Succès :

```
{
  "success": true,
  "data": { ... },
  "message": "Opération réussie"
}
```

Erreur :

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Description de l'erreur",
    "details": { ... }
  }
}
```

Pagination :

```
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  }
}
```

3.3.3 Codes HTTP

- 200 : Succès
- 201 : Créé
- 400 : Mauvaise requête
- 401 : Non authentifié

- 403 : Non autorisé
- 404 : Non trouvé
- 422 : Validation échouée
- 500 : Erreur serveur

3.4 SÉCURITÉ

3.4.1 Authentification JWT

- Génération de token à la connexion (access token)
- Durée de vie : 1h pour access token, 7j pour refresh token
- Stockage : HttpOnly cookie (recommandé) ou localStorage
- Refresh automatique avant expiration
- Révocation des tokens à la déconnexion

3.4.2 Protection des Mots de Passe

- Hachage avec bcrypt (cost factor : 12)
- Politique de mot de passe :
 - Minimum 8 caractères
 - Au moins 1 majuscule, 1 minuscule, 1 chiffre
 - Pas de mots de passe communs (blacklist)

3.4.3 Protection CSRF et XSS

- Tokens CSRF pour opérations sensibles
- Sanitisation des entrées utilisateur
- Headers de sécurité (Helmet.js)
- Content Security Policy (CSP)

3.4.4 Rate Limiting

- Limitation des requêtes API : 100 req/min par IP
- Login : 5 tentatives max puis blocage 15 min
- Endpoints sensibles : limitation stricte

3.4.5 Validation des Données

- Validation côté client ET serveur
- Schémas de validation stricts
- Échappement des caractères spéciaux
- Validation des types et formats

3.4.6 HTTPS

- Certificat SSL/TLS obligatoire en production
- Redirection HTTP → HTTPS
- HSTS (HTTP Strict Transport Security)

3.4.7 Logs de Sécurité

- Enregistrement des tentatives de connexion
- Logs des actions sensibles (admin)
- Alertes sur activités suspectes
- Conservation des logs : 90 jours minimum

3.5 PERFORMANCES

3.5.1 Optimisations Backend

- Mise en cache Redis (optionnel) :
 - Sessions utilisateur
 - Données fréquemment consultées (catalogue)
 - Résultats de recherche
- Pagination systématique
- Requêtes SQL optimisées avec indexes
- Compression gzip des réponses

3.5.2 Optimisations Frontend

- Code splitting (lazy loading)
- Minification et compression des assets
- Images optimisées (WebP si possible)
- Caching navigateur (Cache-Control headers)
- Service Worker pour mode hors ligne (optionnel)

3.5.3 Base de Données

- Indexes sur colonnes fréquemment recherchées
- Partitionnement des tables volumineuses (ventes, mouvements)
- Archivage des données anciennes (> 2 ans)
- Sauvegardes quotidiennes automatiques

3.6 DÉPLOIEMENT

3.6.1 Environnements

- **Développement** : Local
- **Staging** : Serveur de test
- **Production** : Serveur principal

3.6.2 Configuration

Variables d'environnement :

```
DATABASE_URL=postgresql://...  
JWT_SECRET=...  
JWT_REFRESH_SECRET=...  
PORT=3000  
NODE_ENV=production  
FRONTEND_URL=https://...  
SMTP_HOST=... (optionnel)  
SMTP_PORT=...  
SMTP_USER=...  
SMTP_PASS=...
```

3.6.3 Serveur Recommandé

- **Backend** : VPS Linux (Ubuntu/Debian)
- **RAM** : 2GB minimum (4GB recommandé)
- **CPU** : 2 cœurs minimum
- **Stockage** : 20GB SSD minimum
- **Providers** : DigitalOcean, Linode, AWS EC2, Heroku

3.6.4 Base de Données

- PostgreSQL sur le même serveur ou service managé
- Sauvegardes automatiques quotidiennes
- Réplication (optionnel pour haute disponibilité)

3.6.5 CI/CD

- GitHub Actions ou GitLab CI
- Tests automatiques avant déploiement
- Déploiement automatique sur staging
- Déploiement manuel sur production

4. INTERFACES UTILISATEUR

4.1 DESIGN GÉNÉRAL

4.1.1 Principes

- Design moderne et épuré
- Navigation intuitive
- Responsive (desktop prioritaire, mobile secondaire)
- Accessibilité (WCAG 2.1 niveau AA)
- Thème : Clair par défaut (option sombre optionnelle)

4.1.2 Palette de Couleurs (Suggestion)

- Primaire : Bleu (#2563eb) - Actions principales
- Secondaire : Gris (#64748b) - Éléments secondaires
- Succès : Vert (#10b981) - Confirmations
- Avertissement : Orange (#f59e0b) - Alertes
- Erreur : Rouge (#ef4444) - Erreurs
- Neutral : Gris (#f1f5f9) - Fond

4.2 PAGES ET FONCTIONNALITÉS

4.2.1 Page de Connexion

- Formulaire centré
- Champs : Email, Mot de passe
- Bouton "Se connecter"
- Messages d'erreur clairs
- Pas de "Mot de passe oublié" (géré par admin)

4.2.2 Layout Principal (après connexion)

Sidebar (menu latéral) :

- Logo en haut
- Navigation principale :
 -  Dashboard
 -  Livres
 -  Catégories
 -  Ventes
 -  Réductions
 -  Stock (mouvements)
 -  Fournisseurs (admin)

-  Réservations
-  Rapports (admin)
-  Utilisateurs (admin)
-  Notifications
- Profil utilisateur en bas avec déconnexion

Header :

- Titre de la page actuelle
- Barre de recherche globale
- Badge notifications
- Avatar utilisateur

4.2.3 Dashboard

Admin :

- KPI cards :
 - CA du jour
 - Nombre de ventes du jour
 - CA du mois
 - Stock critique (nombre)
- Graphique : Évolution CA (7 derniers jours)
- Liste : 5 livres les plus vendus du mois
- Liste : Livres en stock critique
- Tableau : Performance des vendeurs

Vendeur :

- Mes ventes du jour (nombre et CA)
- Mon CA du mois
- Mes 5 meilleures ventes
- Alertes stock critique

4.2.4 Gestion des Livres

Liste des Livres :

- Tableau avec colonnes :
 - Code
 - Titre
 - Auteur
 - Catégorie

- Stock (avec badge si critique)
- Prix
- Statut
- Actions (Voir/Modifier/Supprimer)
- Filtres :
 - Recherche (titre, auteur, code)
 - Catégorie
 - Statut stock (tous/disponible/critique/rupture)
 - Statut (actif/inactif)
- Pagination
- Boutons : "Ajouter livre", "Importer Excel", "Exporter"

Formulaire Ajout/Modification :

- Tous les champs du livre
- Validation en temps réel
- Boutons : Annuler, Enregistrer

Fiche Détail Livre :

- Toutes les informations
- Historique des ventes
- Historique des mouvements de stock
- Actions rapides : Modifier, Ajuster stock

4.2.5 Gestion des Catégories

Liste :

- Tableau : Nom, Code, Nombre de livres, Statut, Actions
- Recherche
- Bouton "Ajouter catégorie"

Formulaire :

- Nom, Description
- Code généré automatiquement (affiché après saisie du nom)

4.2.6 Point de Vente

Interface de Vente :

- Zone gauche : Recherche et sélection de livres
 - Barre de recherche intelligente

- Résultats en temps réel
- Stock visible pour chaque livre
- Zone droite : Panier
 - Liste des livres ajoutés
 - Quantité modifiable
 - Prix unitaire
 - Réduction auto-appliquée (affichée)
 - Sous-total par ligne
 - Total général
 - Boutons : Vider panier, Valider vente

Après Validation :

- Confirmation de vente
- Affichage facture PDF
- Boutons : Imprimer, Télécharger, Nouvelle vente

4.2.7 Gestion des Réductions

Liste :

- Tableau : Intitulé, Type, Valeur, Période, Statut, Actions
- Filtres : Actives/Expirées/Toutes
- Bouton "Créer réduction"

Formulaire :

- Tous les champs
- Sélection de la cible (globale/livre/catégorie)
- Si livre/catégorie : dropdown de sélection
- Date picker pour début/fin
- Toggle actif/inactif

4.2.8 Mouvements de Stock

Liste :

- Tableau : Date, Livre, Type, Quantité, Stock avant/après, Motif, Utilisateur
- Filtres : Type, Livre, Période, Utilisateur
- Bouton "Ajustement manuel" (admin)

Formulaire Ajustement :

- Sélection livre

- Type : Entrée/Ajustement
- Quantité
- Motif
- Référence (optionnel)

4.2.9 Réservations Clients

Liste :

- Tableau : Client, Téléphone, Livre demandé, Date, Statut, Actions
- Filtres : Statut
- Bouton "Nouvelle réservation"

Formulaire :

- Nom client
- Téléphone
- Livre demandé (texte libre ou sélection)
- Notes
- Boutons : Annuler, Enregistrer

Actions sur réservation :

- Marquer comme disponible
- Convertir en vente
- Annuler

4.2.10 Rapports (Admin)

Page Rapports :

- Sélecteur de type : Journalier/Hebdo/Mensuel/Annuel/Personnalisé
- Si personnalisé : date pickers
- Bouton "Générer"
- Affichage du rapport :
 - KPIs en cards
 - Graphiques
 - Tableaux détaillés
- Boutons : Télécharger PDF, Exporter Excel

Rapports Disponibles :

- Clôture journalière
- Performance période

- Top livres
- Performance vendeurs
- Analyse catégories
- Rotation stock

4.2.11 Gestion Utilisateurs (Admin)

Liste :

- Tableau : Nom, Email, Rôle, Statut, Dernière connexion, Actions
- Bouton "Ajouter utilisateur"

Formulaire :

- Nom complet
- Email
- Rôle (dropdown)
- Mot de passe (à la création uniquement)
- Statut

4.2.12 Notifications

Centre de Notifications :

- Liste des notifications
- Badge non lues
- Filtres : Toutes/Non lues
- Actions : Marquer comme lu, Supprimer

5. LIVRABLES

5.1 CODE SOURCE

- Repository Git (GitHub/GitLab)
- Code commenté et structuré
- README.md complet avec instructions d'installation
- Documentation API (Swagger)

5.2 BASE DE DONNÉES

- Script de création des tables
- Script de données de test (seed)

- Schéma visuel (diagramme ERD)

5.3 DOCUMENTATION

- Cahier des charges (ce document)
- Documentation technique
- Guide utilisateur (admin et vendeur)
- Documentation API

5.4 TESTS

- Tests unitaires (couverture > 70%)
- Tests d'intégration
- Tests E2E sur parcours critiques

5.5 DÉPLOIEMENT

- Scripts de déploiement
- Configuration serveur documentée
- Procédure de sauvegarde/restauration

6. PLANNING ET MÉTHODOLOGIE

6.1 MÉTHODOLOGIE DE DÉVELOPPEMENT

6.1.1 Approche Agile

- Sprints de 2 semaines
- Réunions de suivi régulières
- Livraisons incrémentales
- Tests continus

6.1.2 Workflow Git

```
main (production)
  ↑
staging (pré-production)
  ↑
develop (développement)
  ↑
feature/nom-fonctionnalite
```

6.2 PHASAGE DU PROJET

Phase 1 : Setup et Fondations (Semaine 1-2)

Backend :

- Setup projet (Node.js/Python + Express/FastAPI)
- Configuration base de données PostgreSQL
- Modèles de données (ORM)
- Authentification JWT
- Middleware de sécurité

Frontend :

- Setup projet (React/Vue + Vite)
- Configuration Tailwind CSS
- Structure de dossiers
- Routing
- Authentification côté client

Livrables :

- Environnement de développement fonctionnel
- Connexion/déconnexion opérationnelle
- Structure de base prête

Phase 2 : Gestion du Catalogue (Semaine 3-4)

Backend :

- API Catégories (CRUD)
- API Livres (CRUD complet)
- Génération automatique des codes
- Validation des données
- Gestion du stock de base

Frontend :

- Pages Catégories
- Pages Livres (liste, ajout, modification)
- Composants de formulaires
- Recherche de livres
- Alertes stock critique

Livrables :

- Gestion complète catégories et livres
- Recherche fonctionnelle
- Alertes visuelles stock

Phase 3 : Réductions et Ventes (Semaine 5-6)

Backend :

- API Réductions (CRUD)
- Logique d'application des réductions
- API Ventes
- Génération numéro de facture
- Déduction automatique du stock
- Mouvements de stock automatiques

Frontend :

- Gestion des réductions
- Interface point de vente
- Panier de vente
- Application automatique des réductions
- Validation de vente

Livrables :

- Système de réductions fonctionnel
- Point de vente opérationnel
- Stock mis à jour automatiquement

Phase 4 : Factures et Mouvements (Semaine 7)

Backend :

- Génération de factures PDF
- API Mouvements de stock
- Historique complet
- Ajustements manuels

Frontend :

- Affichage et téléchargement factures
- Page mouvements de stock

- Formulaire d'ajustement manuel
- Historique par livre

Livrables :

- Factures PDF générées
- Traçabilité complète du stock

Phase 5 : Fournisseurs et Commandes (Semaine 8)

Backend :

- API Fournisseurs
- API Commandes fournisseurs
- Réception de commandes
- Mise à jour stock automatique

Frontend :

- Gestion des fournisseurs
- Création de commandes
- Suivi des commandes
- Réception de commandes

Livrables :

- Gestion fournisseurs complète
- Processus de commande fonctionnel

Phase 6 : Rapports et Dashboard (Semaine 9-10)

Backend :

- API Rapports (tous types)
- Calculs statistiques
- Génération PDF rapports
- Export Excel

Frontend :

- Dashboard admin
- Dashboard vendeur
- Pages de rapports
- Graphiques (Recharts)
- Filtres et exports

Livrables :

- Dashboards fonctionnels
- Tous les rapports disponibles
- Exports Excel/PDF

Phase 7 : Réservations et Finalisation (Semaine 11)**Backend :**

- API Réservations
- API Notifications
- API Utilisateurs (gestion)
- Logs d'activité

Frontend :

- Gestion réservations
- Centre de notifications
- Gestion utilisateurs (admin)
- Profil utilisateur

Livrables :

- Réservations opérationnelles
- Notifications fonctionnelles
- Gestion des utilisateurs

Phase 8 : Tests et Optimisation (Semaine 12)

- Tests unitaires backend
- Tests d'intégration
- Tests E2E frontend
- Correction des bugs
- Optimisation des performances
- Amélioration UX
- Documentation finale

Livrables :

- Application stable et testée
- Documentation complète
- Bugs critiques résolus

Phase 9 : Déploiement (Semaine 13)

- Configuration serveur production
- Déploiement backend
- Déploiement frontend
- Configuration base de données
- Sauvegardes automatiques
- Monitoring et logs
- Formation utilisateurs

Livrables :

- Application en production
- Guides utilisateur
- Support et maintenance

6.3 ESTIMATION GLOBALE

- **Durée totale** : 13 semaines (3 mois)
- **Charge de travail** : 1-2 développeurs full-stack
- **Complexité** : Moyenne

6.4 PRÉREQUIS AVANT DÉMARRAGE

- Choix du stack technique validé
- Accès serveur de développement
- Compte GitHub/GitLab
- Maquettes UI approuvées (optionnel)
- Données de test disponibles

7. MAINTENANCE ET ÉVOLUTIONS

7.1 MAINTENANCE CORRECTIVE

- Correction des bugs signalés
- Délai de résolution :
 - Critique : 24h
 - Majeur : 72h
 - Mineur : 1 semaine

7.2 MAINTENANCE ÉVOLUTIVE

- Ajout de nouvelles fonctionnalités
- Améliorations UX
- Optimisations performances

7.3 ÉVOLUTIONS FUTURES (Phase 2)

- Application mobile (React Native/Flutter)
- Programme de fidélité clients
- Gestion multi-magasins
- E-commerce (vente en ligne)
- Intégration systèmes comptables
- Statistiques IA (prédictions de ventes)
- Gestion des employés (pointage, congés)
- Module de caisse enregistreuse

7.4 SUPPORT

- Documentation accessible en ligne
- Formation initiale des utilisateurs
- Support email/téléphone
- Mises à jour régulières

8. CONTRAINTES ET RISQUES

8.1 CONTRAINTES TECHNIQUES

- Compatibilité navigateurs modernes (Chrome, Firefox, Edge, Safari)
- Performance : temps de chargement < 3s
- Disponibilité : 99% uptime
- Sauvegardes quotidiennes obligatoires

8.2 CONTRAINTES FONCTIONNELLES

- Simplicité d'utilisation (vendeurs non techniques)
- Temps de formation < 2h par utilisateur
- Zéro perte de données

8.3 RISQUES IDENTIFIÉS

Risque	Probabilité	Impact	Mitigation
Perte de données	Faible	Critique	Sauvegardes auto + redondance
Défaillance serveur	Moyenne	Élevé	Monitoring + plan de reprise
Bugs critiques	Moyenne	Élevé	Tests rigoureux + QA
Dépassement délai	Moyenne	Moyen	Planning réaliste + buffer
Complexité technique	Faible	Moyen	Stack éprouvé + doc
Résistance utilisateurs	Faible	Moyen	Formation + UX simple

8.4 HYPOTHÈSES

- Connexion internet stable disponible
- Serveur avec ressources suffisantes
- Utilisateurs formés à l'informatique de base
- Données migrables depuis système existant (si applicable)

9. CRITÈRES D'ACCEPTATION

9.1 FONCTIONNELS

✓ Un administrateur peut créer/modifier/supprimer des utilisateurs ✓ Un vendeur peut créer une vente avec réductions automatiques ✓ Les factures PDF sont générées correctement ✓ Le stock est déduit automatiquement à chaque vente ✓ Les alertes s'affichent pour les stocks critiques ✓ Les rapports sont exacts et téléchargeables ✓ Les codes sont générés automatiquement et sont uniques ✓ L'historique des mouvements est complet et traçable ✓ Les commandes fournisseurs mettent à jour le stock ✓ Les réservations peuvent être créées et suivies

9.2 TECHNIQUES

✓ Authentification JWT fonctionnelle et sécurisée ✓ Temps de réponse API < 500ms (95% des requêtes) ✓ Application responsive (desktop + tablette minimum) ✓ Pas d'erreurs console en production ✓ Couverture de tests > 70% ✓ Documentation API complète (Swagger) ✓ Code versionné sur Git ✓ Logs d'erreurs configurés

9.3 SÉCURITÉ

✓ Mots de passe hashés ✓ Tokens JWT avec expiration ✓ Protection CSRF/XSS ✓ Validation des entrées utilisateur ✓ HTTPS en production ✓ Rate limiting actif ✓ Logs de sécurité enregistrés

9.4 PERFORMANCE

✅ Chargement page < 3s ✅ Support de 50+ utilisateurs simultanés ✅ Base de données optimisée avec indexes ✅ Pagination sur toutes les listes ✅ Pas de fuite mémoire

10. GLOSSAIRE

Admin : Utilisateur avec tous les droits de gestion

Vendeur : Utilisateur pouvant effectuer des ventes

JWT : JSON Web Token, système d'authentification

CRUD : Create, Read, Update, Delete (opérations de base)

API REST : Interface de programmation standardisée

Stock critique : Livre dont la quantité $\leq$ seuil minimal

Mouvement de stock : Enregistrement d'une variation de stock

CA : Chiffre d'affaires

UX : Expérience utilisateur

UI : Interface utilisateur

SPA : Single Page Application

ORM : Object-Relational Mapping

PDF : Portable Document Format

CSV : Comma-Separated Values

XAF : Franc CFA (devise)

11. ANNEXES

11.1 Codes d'Erreur Courants

AUTH001 : **Identifiants** invalides

AUTH002 : **Token** expiré

AUTH003 : **Accès** non autorisé

STOCK001 : **Stock** insuffisant

STOCK002 : **Seuil** invalide
VENTE001 : **Livre** introuvable
VENTE002 : **Réduction** invalide
VAL001 : **Données** invalides
VAL002 : **Champ** requis manquant
SYS001 : **Erreur** serveur

11.2 Exemple de Facture

=====

LIBRAIRIE XYZ
123 Rue du Livre
Yaoundé, Cameroun
Tél: +237 123 456 789

=====

Facture N° : FAC-20260105-001
Date : 05/01/2026 14:32
Vendeur : Marie KOUAM

CODE	TITRE	QTÉ	P.U	TOTAL
ROMA123-001	Les Misérables	1	5000	5000
INFO456-012	Python 101	2	8000	16000

Sous-total : 21 000 XAF
Réduction (15%) : -3 150 XAF

TOTAL À PAYER : 17 850 XAF
=====

Merci de votre visite !
=====

11.3 Stack Technique Recommandé (Détailé)

Option 1 : MERN Stack (Recommandé)

- **Frontend** : React 18 + Vite + Tailwind CSS + shadcn/ui
- **Backend** : Node.js + Express.js + TypeScript
- **ORM** : Prisma
- **BDD** : PostgreSQL
- **Auth** : jsonwebtoken + bcrypt
- **PDF** : PDFKit ou jsPDF
- **Charts** : Recharts

- **Validation** : Zod
- **Testing** : Vitest + React Testing Library

Option 2 : PEVN Stack

- **Frontend** : Vue 3 + Vite + Tailwind CSS
- **Backend** : Python + FastAPI
- **ORM** : SQLAlchemy
- **BDD** : PostgreSQL
- **Auth** : python-jose + passlib
- **PDF** : ReportLab
- **Charts** : Chart.js
- **Validation** : Pydantic (natif FastAPI)
- **Testing** : Pytest + Vue Test Utils

12. VALIDATION DU CAHIER DES CHARGES

Date de rédaction : Janvier 2026 **Version** : 1.0 **Statut** :  En attente de validation

Signatures :

- Chef de projet : _____
- Client : _____
- Développeur : _____

FIN DU CAHIER DES CHARGES